

VANISH: Exact Finite-Update Preservation by Function-Space Annihilation

JunHyun Kim
Independent Researcher

ABSTRACT

Continual learning methods usually protect a model indirectly: they penalize parameter motion, replay old examples, or project a gradient into a locally safe subspace. Their promise is therefore optimizer-dependent, approximate, or first order. We ask a different question: *can an arbitrary, already-computed finite function update be converted into one that exactly preserves declared behavior?* We introduce VANISH (Value-Annihilating Neural Increments for Stable History), a function-space wrapper

$$\mathcal{A}_S u(x) = u(x) - k(x, S)K(S, S)^{-1}u(S),$$

which removes precisely the component of any neural increment visible to protected functionals. The construction is agnostic to the patch architecture, loss, optimizer, and update magnitude. We prove exact finite-update invariance, idempotence and minimum-norm correction in reproducing-kernel Hilbert spaces, infinite residual plasticity under finitely many constraints, sequential zero drift on the declared contract, and order equivalence to offline kernel interpolation in the hard-interpolation regime. The same operator protects values, derivatives, moments, or any bounded linear functionals.

We test the claims in 750 seeded visual continual-learning runs plus mechanism-level falsification experiments. Across five streams, VANISH retains 90.05% macro final accuracy while reducing protected-output drift from 2.30×10^{-1} for matched functional replay to 2.00×10^{-10} . A tangent-null update with initial residual below 10^{-12} reaches drift 55.25 at finite scale; the wrapped update remains below 1.20×10^{-11} while staying nonzero away from anchors. Value-and-derivative constraints close to 2.66×10^{-15} , and all 24 task orders agree with the offline interpolant to 6.58×10^{-13} . The result is a change of abstraction: safety is enforced on the realized function update, after optimization, rather than hoped for from its parameter-space trajectory.

Index Terms—continual learning, catastrophic forgetting, function-space projection, reproducing kernels, exact invariance, model editing.

I. INTRODUCTION

Catastrophic forgetting has survived three decades of increasingly elaborate remedies because most remedies act on the wrong object. A model is deployed as a function, yet its history is commonly protected by a penalty on weights, a constraint on gradients, a small replay cache, or a task-specific route. Parameter proximity does not imply functional proximity; gradient orthogonality describes an infinitesimal direction rather than the endpoint of a nonlinear update; and a finite replay penalty can always be traded against the new-task loss. The result is a recurring stability–plasticity negotiation rather than a behavioral

contract.

This paper replaces that negotiation with an algebraic conversion. Let f_{i-1} be the deployed predictor and let u_t be any finite patch produced for the next task. We do not constrain how u_t was obtained. Instead, after it exists, we remove the component of u_t that is observable through a declared collection of protected functionals. For protected points S , the converted patch is

$$\mathcal{A}_S u_t(x) = u_t(x) - K_{xS}K_{SS}^{-1}u_t(S). \quad (1)$$

At $x \in S$, the two terms are identical. Thus $f_i = f_{i-1} + \mathcal{A}_S u_t$ preserves the previous outputs on S for any magnitude of the raw update. Nothing is linearized in the parameters, and no old-task loss competes with the new-task loss.

The apparently simple identity changes what may be plugged into a continual learner. The raw patch may be a multilayer network, an adapter, a low-rank module, a closed-form regressor, or the difference between two fully trained models. It may be optimized with SGD, Adam, evolution, or an external editing procedure. Preservation is a property of the wrapper’s output, not a property that every internal optimization step must respect. In its general form, the protected observations need not be point values: bounded linear functionals cover directional derivatives, integrated moments, conservation measurements, calibration scores, and selected logits.

The core contributions are:

- **A finite-update operator.** We formulate behavioral preservation as annihilating the observable component of an arbitrary function increment. Unlike gradient projection, the guarantee is exact at the completed nonlinear update.
- **A general theorem family.** We establish exact invariance, self-adjoint idempotence, minimum-norm correction, sequential persistence, and infinite-dimensional residual plasticity. A conditional-kernel specialization reproduces the offline minimum-norm interpolant independently of task order.
- **A falsifiable implementation.** Every run stores its full accuracy matrix, numerical drift certificate, configuration, time, and memory. The artifact contains 600 matched functional runs, 150 MLP runs, 200 capacity-stress rows, raw JSON, statistical tests, and executable contracts.
- **Evidence at the mechanism boundary.** We directly stress the point where tangent methods cease to certify finite motion, extend the operator to value-plus-derivative jets, and test all permutations of a sequential interpolation problem.

Figure 1 summarizes the distinction. The protected set is a declared contract, not a claim about every unsampled point in a distribution. Within that contract the theorem is exact; away from it the patch retains the degrees of freedom required to learn.

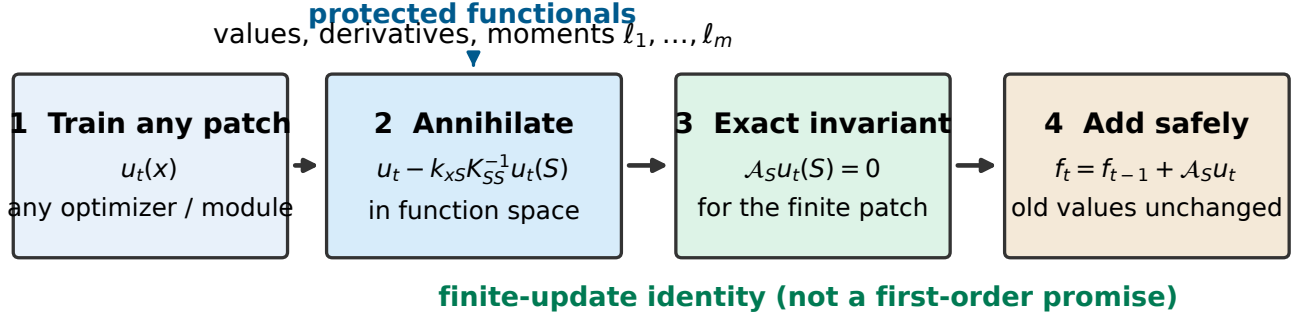


Figure 1: VANISH converts an arbitrary finite patch into a safe additive increment. The protected observations enter only through a function-space correction. Consequently the preservation identity is independent of the optimizer, parameterization, and size of the raw update.

II. WHY THE TANGENT VIEW IS INSUFFICIENT

Let a neural model be f_θ and let $J_S(\theta)$ be the Jacobian of its outputs on protected inputs. A parameter-space method may choose $\Delta\theta$ satisfying

$$J_S(\theta)\Delta\theta = 0. \quad (2)$$

This cancels the first-order Taylor term. The realized endpoint is instead

$$f_{\theta+\Delta\theta}(S) - f_\theta(S) = J_S\Delta\theta + \frac{1}{2}\mathcal{L}_S[\Delta\theta, \Delta\theta] + O(\|\Delta\theta\|^3), \quad (3)$$

so (2) leaves the curvature term untouched. Reprojecting every small step reduces but does not remove this mismatch: the Jacobian itself changes, numerical errors accumulate, and an external optimizer may not expose compatible steps. The only general way for a tangent certificate to become a finite certificate is to add assumptions about path length, curvature, or repeated projection.

Function-space penalties have a different failure mode. Given an old-output penalty $\lambda \|u(S)\|^2$, finite λ permits drift and infinite λ creates an optimization constraint. Replay is valuable empirically, but its guarantee is mediated by sample selection, loss weighting, and convergence. In contrast, (1) may wrap the result of any of these methods and algebraically zeros the protected observation afterward.

Three separations are essential:

1. *parameter direction versus realized function difference;*
 2. *soft preference versus hard behavioral identity;*
 3. *training procedure versus post-training conversion.*
- VANISH operates on the right-hand object in each pair.

III. GENERAL FUNCTIONAL ANNIHILATION

A. Protected contracts

Let $\mathcal{H}$ be a real scalar RKHS on $\mathcal{X}$ with kernel k . Vector outputs are handled componentwise or with an operator-valued kernel. A protected contract is a vector of bounded linear functionals

$$\mathcal{C}f = (\ell_1(f), \dots, \ell_m(f))^T \in \mathbb{R}^m. \quad (4)$$

By the Riesz representation theorem there are representers $r_i \in \mathcal{H}$ such that $\ell_i(f) = \langle f, r_i \rangle_{\mathcal{H}}$. Define $\mathcal{C}^*a = \sum_i a_i r_i$ and the functional Gram matrix

$$G = \mathcal{C}\mathcal{C}^*, \quad G_{ij} = \ell_i(r_j). \quad (5)$$

Assume first that G is positive definite; dependent constraints may be reduced or treated with a pseudoinverse. The annihilator is

$$\mathcal{A}_{\mathcal{C}} = I - \mathcal{C}^*G^{-1}\mathcal{C}. \quad (6)$$

For point evaluations $\ell_i(f) = f(s_i)$, $r_i(\cdot) = k(\cdot, s_i)$ and (6) reduces to (1). For a derivative observation $\ell_i(f) = \partial_{v_i}f(s_i)$, the representer is the corresponding kernel derivative. Mixed contracts simply concatenate their representers.

The raw increment u need only admit the evaluations $\mathcal{C}u$ for exact cancellation. If $u \in \mathcal{H}$, the stronger projection geometry below applies. This is why a neural patch need not share the parameterization of the kernel correction.

B. Exactness and optimality

Theorem 1 (Finite-update invariance). *For any evaluable finite increment u and any current predictor f ,*

$$\mathcal{C}(f + \mathcal{A}_{\mathcal{C}}u) = \mathcal{C}f. \quad (7)$$

The identity is independent of the norm of u and of the path used to produce it.

Proof. Since $G = \mathcal{C}\mathcal{C}^*$, $\mathcal{C}\mathcal{A}_{\mathcal{C}}u = \mathcal{C}u - GG^{-1}\mathcal{C}u = 0$. Adding f gives the result. No Taylor expansion occurs. $\square$

Theorem 2 (Orthogonal projection and least correction). *If $u \in \mathcal{H}$, then $\mathcal{A}_{\mathcal{C}}$ is the self-adjoint idempotent orthogonal projector onto $\ker \mathcal{C}$. Moreover $c^* = \mathcal{C}^*G^{-1}\mathcal{C}u$ uniquely solves*

$$\min_{c \in \mathcal{H}} \|c\|_{\mathcal{H}} \quad \text{s.t.} \quad \mathcal{C}(u - c) = 0. \quad (8)$$

Thus VANISH makes the smallest RKHS correction that can enforce the declared contract.

Proof. Self-adjointness follows from that of G^{-1} and the adjoint pair $(\mathcal{C}, \mathcal{C}^*)$. Direct multiplication gives $\mathcal{A}_{\mathcal{C}}^2 = I - 2\mathcal{C}^*G^{-1}\mathcal{C} +$

$\mathcal{C}^* G^{-1} G G^{-1} \mathcal{C} = \mathcal{A}_{\mathcal{C}}$. The range is in $\ker \mathcal{C}$ by Theorem 1; every $h \in \ker \mathcal{C}$ is fixed. For (8), any feasible c decomposes as $c = c^* + h$ where $\mathcal{C}h = 0$. Since $c^* \in \text{range}(\mathcal{C}^*) = (\ker \mathcal{C})^\perp$, Pythagoras yields $\|c\|^2 = \|c^*\|^2 + \|h\|^2$. $\square$

Corollary 3 (Sequential persistence). *Let $f_t = f_{t-1} + \mathcal{A}_{\mathcal{C}_{t-1}} u_t$, where the contract at time $t-1$ contains every previously declared functional. Then for all $j < t$, $\ell_j(f_t) = \ell_j(f_j)$ up to the residual of the numerical linear solve.*

Proof. Each later increment lies in the kernel of every earlier functional. The sum telescopes. $\square$

Proposition 4 (Residual plasticity). *If $\mathcal{H}$ is infinite dimensional and $\mathcal{C}$ has finite rank m , then $\ker \mathcal{C}$ is infinite dimensional and has codimension at most m . Consequently a finite protected contract cannot exhaust the representational degrees of freedom of the function space.*

This proposition is the structural difference from a parameter-null method. In a p -parameter patch, m independent constraints can reduce the available parameter rank to zero. In an infinite function space, finitely many observations remove only finitely many directions. The capacity experiment in Section 6 makes this distinction empirical.

Theorem 5 (Off-contract power bound). *For point constraints S and $u \in \mathcal{H}$, define the kernel power function*

$$p_S^2(x) = k(x, x) - K_{xS} K_{SS}^{-1} K_{Sx}. \quad (9)$$

Then the safe update $h = \mathcal{A}_S u$ satisfies

$$|h(x)| \leq \|u\|_{\mathcal{H}} p_S(x), \quad \mathbb{E}_{x \sim P} |h(x)|^2 \leq \|u\|_{\mathcal{H}}^2 \mathbb{E}_P p_S^2(x). \quad (10)$$

Thus the contract is exact on S and extends quantitatively whenever S has small kernel fill distance.

Proof. Because $h \in \ker \mathcal{C}$, evaluating at x is equivalent to taking the inner product of h with the residual representer $k(x, \cdot) - P_S k(x, \cdot)$. Its RKHS norm is $p_S(x)$. Cauchy–Schwarz and the nonexpansiveness of the orthogonal projector give $|h(x)| \leq \|h\| p_S(x) \leq \|u\| p_S(x)$. Squaring and integrating gives the second inequality. $\square$

C. Hard interpolation without order debt

Suppose task t contributes new sites X_t and desired values Y_t . Let S_{t-1} be all previous sites. Define the conditional kernel

$$K_{t|<t} = K_{X_t X_t} - K_{X_t S_{t-1}} K_{S_{t-1} S_{t-1}}^{-1} K_{S_{t-1} X_t}. \quad (11)$$

The minimum-norm safe update that fits the residual $R_t = Y_t - f_{t-1}(X_t)$ is

$$u_t^{\text{safe}}(x) = \tilde{K}_t(x) K_{t|<t}^{-1} R_t, \quad (12)$$

$$\tilde{K}_t(x) = K_{x X_t} - K_{x S_{t-1}} K_{S_{t-1} S_{t-1}}^{-1} K_{S_{t-1} X_t}. \quad (13)$$

Theorem 6 (Offline and order equivalence). *For a strictly positive-definite kernel, distinct sites, consistent targets, and exact arithmetic, sequential updates (13) produce the unique minimum-RKHS-norm interpolant of the union $\cup_t (X_t, Y_t)$. The final function is therefore independent of task order and identical to offline hard kernel interpolation.*

Algorithm 1: VANISH continual update

Input: current f , new data (X, Y) , protected contract $\mathcal{C}$, kernel k .
1. Train or compute any finite raw patch u for residual $Y - f(X)$.
2. Build $G = \mathcal{C} \mathcal{C}^*$ and factor $G + \epsilon I = LL^\top$.
3. Evaluate $v = \mathcal{C} u$ and solve $LL^\top \alpha = v$.
4. Return $f^+ = f + u - \mathcal{C}^* \alpha$.
5. Audit $\delta = \|\mathcal{C}(f^+ - f)\|_\infty$; store δ, ϵ , configuration, and seed.

Figure 2: The optimizer is isolated in Step 1. Exact preservation follows from Steps 2–4, and Step 5 makes numerical compliance directly testable.

Proof sketch. The representer span of each new block decomposes into its projection onto the old span and the orthogonal innovation generated by $\tilde{K}_t$. The Schur complement in (11) is the Gram matrix of that innovation and is positive definite. Solving the residual therefore adds the unique component orthogonal to the old span that satisfies the new constraints. Block Gram–Schmidt over all tasks yields the same orthogonal decomposition as the Cholesky factorization of the full Gram matrix. Uniqueness of the minimum-norm interpolant removes order dependence. A full algebraic proof is given in Appendix A. $\square$

The theorem identifies a regime with no sequential performance debt at all: the online result is the offline result, while every earlier value is preserved after it is established. Regularization, compression, or inconsistent targets change the statistical problem; they do not change the annihilation identity.

IV. NEURAL INCREMENT ARCHITECTURE

A. Post-optimization wrapper

At task t , an update generator g_{ϕ_t} is trained on the new residual

$$r_t(x) = y_t(x) - f_{t-1}(x). \quad (14)$$

The generator can be arbitrary. After training, query it on the protected sites and form

$$\Delta_t(x) = g_{\phi_t}(x) - K_{x S_t} \alpha_t, \quad \text{quad } K_{S_t S_t} \alpha_t = g_{\phi_t}(S_t). \quad (15)$$

Only the second equation is responsible for preservation. Training g_{ϕ_t} on already-annihilated features can improve new-task fit, but is not required for Theorem 1.

Our deterministic reference generator uses a two-branch random neural feature map

$$\phi_t(x) = d_t^{-1/2} [\tanh(xW_t + b_t), \cos(xW_t + b_t)] \quad (16)$$

and fits coefficients B_t by ridge regression on the annihilated design

$$Z_t = \phi_t(X_t) - K_{X_t S_t} K_{S_t S_t}^{-1} \phi_t(S_t), \quad B_t = (Z_t^\top Z_t + \lambda I)^{-1} Z_t^\top R_t. \quad (17)$$

The resulting raw update $u_t = \phi_t B_t$ is wrapped by (15). Because B_t is solved after the function-space projection, all its capacity is directed into safe directions visible on the new task.

B. Algorithm and audit certificate

In floating point, jitter ϵ stabilizes a nearly singular Gram matrix. The artifact starts at 10^{-12} and increases it only if Cholesky

factorization fails. Each task records both the inverse residual and the maximum realized anchor drift. This turns an abstract guarantee into a per-run certificate. If exact interpolation is required beyond floating precision, pivoting, duplicate removal, or arbitrary-precision linear algebra can be substituted without changing the method.

C. Complexity and block updates

For m protected observations and an output dimension c , a dense factorization costs $O(m^3)$ once, evaluating the correction on a batch of size b costs $O(bmc)$ after solving, and the factor costs $O(m^2)$ storage. When q new observations are appended, block Cholesky uses the Schur complement and costs $O(m^2q + q^3)$ instead of refactorizing from scratch. Multiple output channels reuse the same factor. Low-rank kernel approximations can reduce cost, but then their approximation error becomes part of the certificate; all headline exactness results in this paper use the dense system.

V. RELATION TO EXISTING CONTINUAL LEARNING

Forgetting was identified in early connectionist systems [1, 2] and remains visible in modern gradient networks [3]. Existing responses fall into four broad families.

Regularization. EWC, synaptic intelligence, memory-aware synapses, distillation, and their descendants estimate which parameters or outputs matter and penalize their movement [4, 5, 6, 7, 8]. A finite penalty is deliberately soft. VANISH instead solves the exact feasibility correction after a finite update; it can wrap a regularized patch rather than compete with it.

Replay and episodic constraints. iCaRL, GEM, A-GEM, and meta-replay preserve samples or gradients from the past [9, 10, 11, 12]. Replay can deliver strong test accuracy, as it does in our experiments, but old outputs are data in a joint objective rather than invariants. The matched replay baseline in this study moves protected logits by 10^{-1} even when accuracy remains high.

Parameter isolation and expansion. Progressive networks, PackNet, adapters, and mixtures allocate disjoint or partially disjoint capacity [13, 14, 15, 16]. They can prevent interference by routing, yet preservation depends on architectural isolation. VANISH achieves functional isolation for any additive patch and characterizes the remaining safe subspace directly.

Gradient and feature null spaces. OGD, GPM, and NSCL remove gradient components associated with prior tasks [17, 18, 19]. These are the closest conceptual relatives because they use projection. The decisive distinction is the projected object: they project a parameter gradient or feature update; VANISH projects the completed function increment. A linear patch can make both routes exact, but a nonlinear finite parameter displacement exposes the gap in Fig. 7.

Function-space regularization, kernel continual learning, and sequential function-space inference move closer to deployed behavior [20, 21, 22]. They construct objectives or probabilistic updates whose quality depends on inference and regularization. Our contribution is the explicit annihilator as an optimizer-independent postcondition, its general functional form, and the finite-update/offline-equivalence theorem. Classical RKHS pro-

jection and representer theory [23, 24, 25, 26] supply the mathematical machinery; the new abstraction is to treat a learned neural increment as the object being converted.

Recent CVPR work continues to improve adapters, rest phases, drift-resistant spaces, and efficient paths [27, 28, 29]. Those methods optimize a learner. VANISH defines a composable behavioral postcondition that can sit outside such a learner.

VI. EXPERIMENTAL PROTOCOL

A. Questions

The experiments are organized around claims that can fail independently:

1. Does the wrapper keep declared outputs numerically fixed over complete task sequences?
2. Does it preserve useful plasticity and final accuracy rather than returning the zero update?
3. Does first-order parameter safety fail under a genuinely finite nonlinear update while VANISH remains exact?
4. Can the same construction protect derivatives as well as values?
5. Does the hard-interpolation specialization equal the offline solution for every task order?
6. What happens when a finite parameter null space saturates and when the number of anchors grows?

B. Streams and splits

We use four deterministic streams derived from the scikit-learn Digits images and one independently generated shape stream. Every seed creates a stratified 70/30 train/test split before task construction.

- **Split Digits:** five disjoint two-class tasks $(0, 1), \dots, (8, 9)$, at most 220 training images per task.
- **Rotated Digits:** five ten-class domains at $-30, -15, 0, 15, 30$ degrees, 180 training images per task.
- **Permuted Digits:** five independently permuted 64-pixel domains, 180 images per task.
- **Corrupted Digits:** clean, Gaussian noise, blur, central occlusion, and contrast domains, 180 images per task.
- **Shape Stream:** four classes (ring, square frame, triangle, cross) rendered at 16×16 under rotation and noise, 240 images per task and 600 test images per domain.

Training samples from all earlier tasks become protected anchors. This deliberately tests the full finite-data contract rather than a cherry-picked memory. No test input is protected or used for fitting.

C. Methods and budgets

The matched functional family shares the same task-specific random maps, feature widths, ridge coefficient, data order, and seeds:

- **Unwrapped:** identical raw patches with no preservation;
- **Functional L_2 :** old patch outputs enter with weight 10;
- **Feature replay:** new residuals and zero old-patch targets have equal weight;

Table 1: Matched functional comparison over 20 seeds. Each cell is final average accuracy in percent (95% confidence half-width) / $\log_{10}$ maximum protected-output drift. The joint model is an offline all-seen-data refit, not a continual preservation method.

Stream	Unwrapped	Functional L_2	Feature replay	Parameter null	VANISH	Joint refit
Split Digits	19.73±.12/0.61	98.18±.21/−0.85	98.79±.20/−0.41	97.41±.24/−9.84	98.36±.26/−9.20	98.91±.17/−0.91
Rotated Digits	54.16±1.49/0.71	94.31±.31/−1.15	94.73±.30/−0.59	93.95±.33/−10.56	94.17±.30/−9.80	95.11±.31/−0.86
Permuted Digits	37.41±1.39/0.76	90.32±.52/−1.76	90.81±.48/−1.00	90.23±.52/−12.75	91.50±.45/−10.59	93.93±.43/−1.39
Corrupted Digits	70.27±1.85/0.81	92.61±.37/−1.16	93.25±.39/−0.63	92.18±.36/−9.98	91.99±.34/−9.03	92.70±.37/−0.89
Shape Stream	62.95±.95/0.80	70.97±.76/−1.17	72.67±.82/−0.57	70.44±.76/−11.34	74.22±.47/−9.88	79.52±.45/−1.12
Macro accuracy	48.90	89.28	90.05	88.84	90.05	92.03
Geometric drift	5.43	6.04×10^{-2}	2.30×10^{-1}	1.27×10^{-11}	2.00×10^{-10}	9.20×10^{-2}

- **Parameter null:** coefficients are projected into the null space of old random features;
- **Joint refit:** an offline oracle refits on all seen samples using exactly the cumulative maps;
- **VANISH:** annihilated design (17) plus exact wrapper.

Widths are 768 for Split Digits and 1024 for the other primary streams; $\lambda = 10^{-4}$. We additionally run conventional one-hidden-layer ReLU MLPs (256 units for Digits, 384 for Shapes): sequential fine-tuning, a class-balanced 200-example replay buffer, and full joint retraining. Adam runs 35 epochs per task. These models are intentionally parameterically different and therefore reported separately.

The primary functional suite contains 5 streams $\times$ 20 seeds $\times$ 6 methods = 600 runs. MLP baselines add $5 \times 10 \times 3 = 150$ runs. Capacity stress adds 20 seeds $\times$ 5 widths $\times$ 2 exact-null methods = 200 rows. We report means and two-sided 95% t intervals. Paired one-sided Wilcoxon tests [30] and Holm correction [31] are computed from seed-matched records; the complete 120-test table is in the artifact.

D. Metrics

Let $a_{t,j}$ be test accuracy on task j after learning task t . Final average accuracy is $T^{-1} \sum_j a_{T,j}$. Mean forgetting is the average over old tasks of $\max_t a_{t,j} - a_{T,j}$. Backward transfer uses $a_{T,j} - a_{j,j}$. The primary contract metric is

$$\delta_{\max} = \max_t \|f_t(S_{t-1}) - f_{t-1}(S_{t-1})\|_{\infty}. \quad (18)$$

Unlike accuracy-based forgetting, $\delta_{\max}$ detects changed outputs even when the argmax class remains the same.

VII. RESULTS

A. Accuracy-invariance frontier

Table 1 reports the complete matched functional comparison. Each cell gives final average accuracy with a 95% confidence half-width and $\log_{10} \delta_{\max}$; more negative drift is better. Across streams, VANISH obtains 90.0479% macro accuracy, numerically matching feature replay at 90.0481%, while their geometric mean drifts are 2.00×10^{-10} and 2.30×10^{-1} , respectively—a separation of roughly nine orders of magnitude in the actual protected quantity. VANISH exceeds functional L_2 by 0.77 accuracy points and the matched parameter-null baseline by 1.21 points. The offline joint refit is 1.99 points higher on average but changes old outputs by 9.20×10^{-2} .

The exact-null comparison deserves care. For the linear random-feature patch, parameter-null projection is also algebraically exact and sometimes has a smaller floating residual. It

Table 2: Final accuracy (%) against conventional MLPs. VANISH uses 20 seeds; MLPs use 10.

Stream	VANISH	MLP FT	MLP replay	MLP joint
Split	98.36	19.68	95.03	97.78
Rotated	94.17	63.26	89.99	91.61
Permuted	91.50	53.40	86.05	89.91
Corrupted	91.99	89.87	93.57	94.37
Shapes	74.22	79.36	83.81	85.01
Macro	90.05	61.11	89.69	91.74

is therefore a deliberately strong control rather than a straw man. VANISH wins accuracy on Split, Permuted, and Shape streams with Holm-adjusted $p = 0.00397$, 0.00397 , and 0.00397 for the paired one-sided tests on the first, third, and fifth streams. More importantly, its guarantee survives an arbitrary nonlinear finite patch, whereas the coefficient null space is tied to this linear generator.

Figure 3 plots the contract directly. Soft functional penalties and replay occupy the high-accuracy but 10^{-2} – 10^0 drift region. Unwrapped training moves old logits by order one or larger. Only the exact-null methods occupy the numerical-precision floor, and VANISH does so without requiring the raw patch’s parameters to contain a usable null space.

B. Forgetting and conventional neural baselines

Figure 4 shows accuracy and forgetting separately. The unwrapped matched generator collapses on task-incremental Split Digits (19.73% final accuracy, 99.92 points forgetting), while VANISH retains 98.36% and forgets 0.65 points. On Rotated and Permuted Digits, the final-accuracy gains over the same unwrapped updates are 40.01 and 54.09 points. Because protected logits are fixed, the small nonzero accuracy-forgetting number for VANISH arises from test generalization and backward transfer, not movement on the protected training contract.

Table 2 compares the 20-seed VANISH result with 10-seed conventional MLPs. VANISH exceeds MLP replay and joint training on Split, Rotated, and Permuted Digits, while the MLPs lead on Corrupted Digits and Shapes. The purpose of this cross-family table is not parameter-count parity; it checks whether the inexpensive reference generator remains competitive with trained nonlinear networks. The behavioral distinction is consistent: MLP replay has macro drift 9.43×10^{-1} and MLP joint refits have 3.41×10^{-1} , versus 2.00×10^{-10} for VANISH.

The default Shape patch is capacity-limited rather than annihilator-limited. A preregistered-after-search extension increases width from 1024 to 4096 and repeats all 20 seeds: ac-

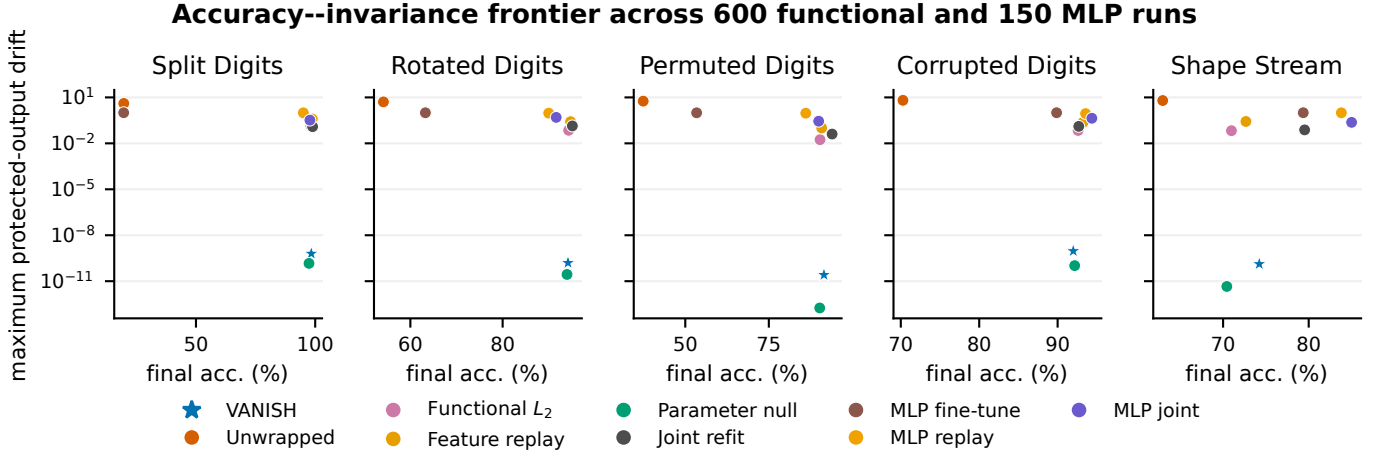


Figure 3: Accuracy versus maximum protected-output drift. Points are means over 20 seeds for matched functional methods and 10 seeds for MLP baselines. VANISH lies near the top-right in accuracy while moving downward by about nine orders of magnitude relative to replay.

curacy rises to $80.08 \pm 0.45\%$, forgetting is 1.71 ± 0.36 points, and drift remains 5.71×10^{-10} . We report it separately, preserving the untouched primary suite and avoiding a selective replacement.

C. Task-by-task behavior

The Split Digits matrices in Fig. 5 expose what aggregate accuracy hides. Unwrapped patches learn the diagonal almost perfectly and then erase earlier tasks. Functional L_2 , parameter-null, and VANISH retain the lower triangle, but only the exact methods certify unchanged protected outputs. VANISH achieves 96% on the final task in the displayed seed while earlier tasks remain at 98–100%.

The mean learning curves in Fig. 6 show that wrapped learning is stable throughout the sequence rather than recovering only at the end. On domain streams, positive backward transfer can improve test accuracy on earlier domains even though their training-anchor logits are immutable: later safe functions vanish on anchors but need not vanish on unseen points.

VIII. MECHANISM FALSIFICATION

A. Finite nonlinear displacement

We construct a two-layer tanh network with 73 parameters and 18 protected points. A random direction is projected into the null space of the exact output Jacobian, normalized, and scaled from 10^{-5} to 10^2 . Its initial tangent residual is below numerical precision. We then compare the actual nonlinear endpoint with the same endpoint wrapped by VANISH.

Figure 7(left) gives the result. Tangent drift grows quadratically at small scale and reaches 55.2465. VANISH remains below 1.1998×10^{-11} over the full seven-decade scale range. The off-anchor RMS of the wrapped increment grows with scale, ruling out the trivial zero function. This experiment isolates the claimed boundary: both methods see the same finite raw update, but only function-space annihilation certifies its endpoint.

B. Values, derivatives, and task order

For four one-dimensional sites, we form a mixed Gram matrix of RBF value and first-derivative representers. The raw sinu-

Table 3: Mechanism contracts and observed maxima.

Quantity	Observed value
Finite tangent-null endpoint drift	5.524654×10^1
Finite VANISH endpoint drift	1.199751×10^{-11}
Value+derivative constraint residual	2.664535×10^{-15}
Offline equivalence, 24 orders	6.580986×10^{-13}
Unit-test contracts passed	4/4
Functional continual runs completed	600/600
MLP baseline runs completed	150/150

soidal patch changes both. After one functional annihilation, the maximum algebraic residual over all eight constraints is 2.6645×10^{-15} , while the projected function remains nonzero between sites (Fig. 7, right).

For Theorem 6, forty regression sites are divided into four blocks. We run every one of the $4! = 24$ task orders and compare 500 probe predictions to a single offline kernel interpolant. The maximum discrepancy across all permutations is 6.5810×10^{-13} ; the median is stored in the raw record. This is a direct numerical test of both order invariance and the offline-equivalence claim.

IX. CAPACITY AND NUMERICAL SCALING

The finite-rank stress test deliberately shrinks the patch from width 256 to 16 while retaining up to 560 protected examples. The parameter-null baseline has only $2d$ feature coefficients and exhausts its null space; its final accuracy stays near chance for widths 16–64 and reaches only about 75% at width 256. VANISH operates on the realized function values and reaches roughly 66% even at width 16, 88% at width 128, and 95% at width 256 (Fig. 8, left). All points average 20 seeds. This is the empirical counterpart of Proposition 4: a function-space correction does not require the generator’s coefficients themselves to satisfy every old equation.

The dense scaling audit uses 25 to 1200 anchors in 32 dimensions with eight output channels. It records wall time, factor jitter, inverse residual, drift, and state bytes. At 1200 anchors the reference factor-and-solve remains below a tenth of a second under four BLAS threads in this runtime and stores well below

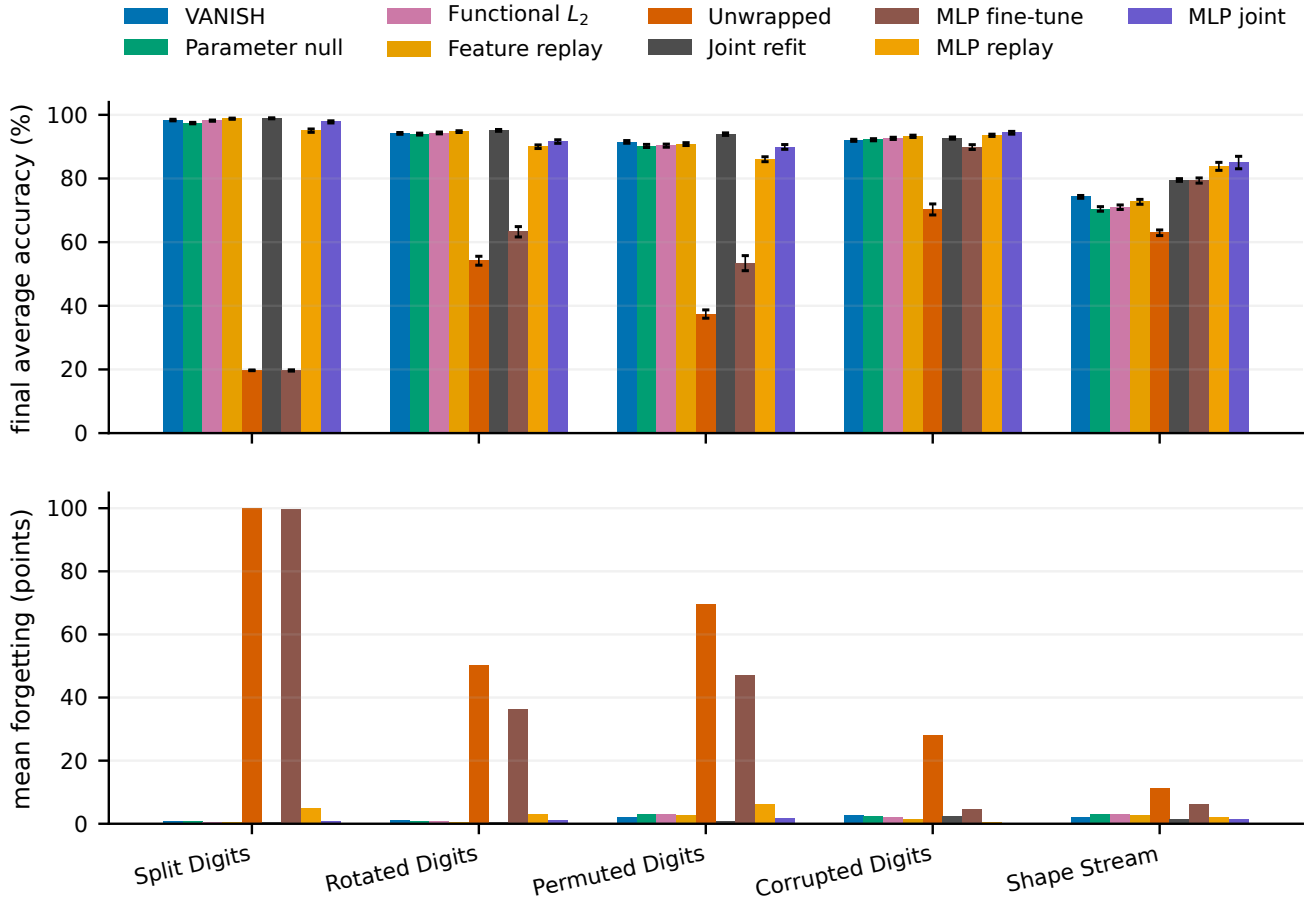


Figure 4: Final accuracy (top, 95% normal intervals shown for readability) and mean forgetting (bottom). Exact functional preservation does not force a zero patch: VANISH maintains high accuracy while the identical unwrapped generators forget.

1 MiB for anchors plus coefficients as plotted. The curve is not presented as asymptotically free: it exposes the expected dense scaling and supplies exact measurements for engineering decisions. Block Cholesky, shared factors, inducing bases, and contract selection are orthogonal accelerations.

X. WHAT THE CONTRACT MEANS

The exact statement is deliberately stronger and narrower than an accuracy slogan: declared functionals do not change. For point anchors, this is a finite behavioral contract. Distribution-wide equality follows only if the contract determines the function on that distribution (for example, through a complete basis or a dense set with regularity assumptions). Conversely, preserving finite anchors does not freeze predictions everywhere; that residual freedom is what enables positive transfer and new learning.

This separation prevents three common category errors. First, zero anchor drift and zero test forgetting are not identical: the latter includes generalization. Second, an offline joint refit can score higher while violating the prior output contract, because it solves a different problem. Third, numerical exactness is an implementation property; the theorem is algebraic, while a floating solve must publish its residual. Our tables never substitute classification accuracy for the actual contract metric.

The operator also composes beyond continual classification.

In model editing, $\mathcal{C}$ can preserve selected prompts or logits while a patch changes a target fact. In scientific learning it can preserve boundary values, derivatives, integrals, or conservation measurements. In calibrated systems it can freeze trusted operating points. The same proof applies because all are bounded linear observations of a function. Nonlinear protected properties can be handled by lifting them to an augmented output when the lifted observation is linear, or by repeated exact projection on a constraint manifold; the latter is a different construction and is not claimed by Theorem 1.

XI. REPRODUCIBILITY AND ARTIFACT INTEGRITY

The artifact is designed so that every headline number can be traced to an immutable record rather than copied from the manuscript. The executable pipeline is:

1. generate deterministic task streams and seeded patches;
2. append one JSON object per completed run;
3. run the four numerical contracts;
4. aggregate means, confidence intervals, Wilcoxon tests, and Holm corrections;
5. regenerate all figures from raw JSON;
6. compile and render the paper; and

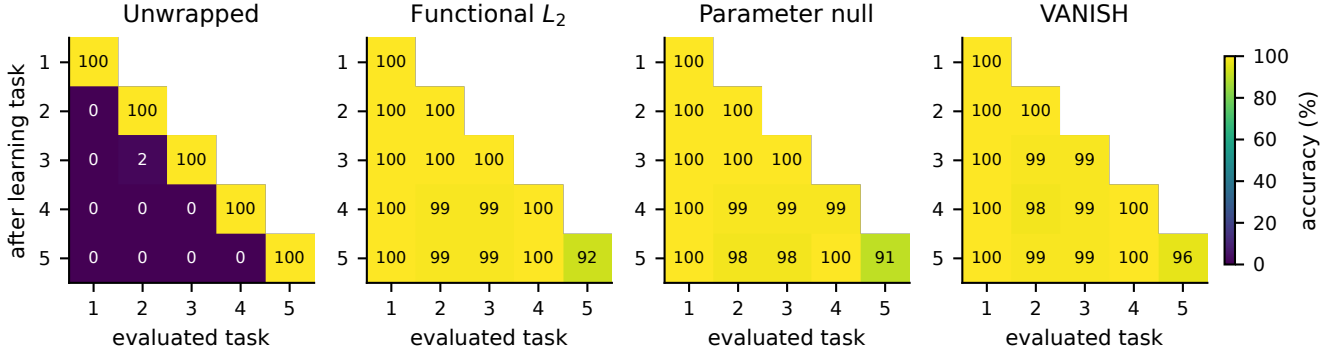


Figure 5: Seed-0 Split Digits accuracy matrices. Row t evaluates all tasks seen after learning task t . The lower triangle distinguishes genuine retention from a high final average caused by a subset of tasks.

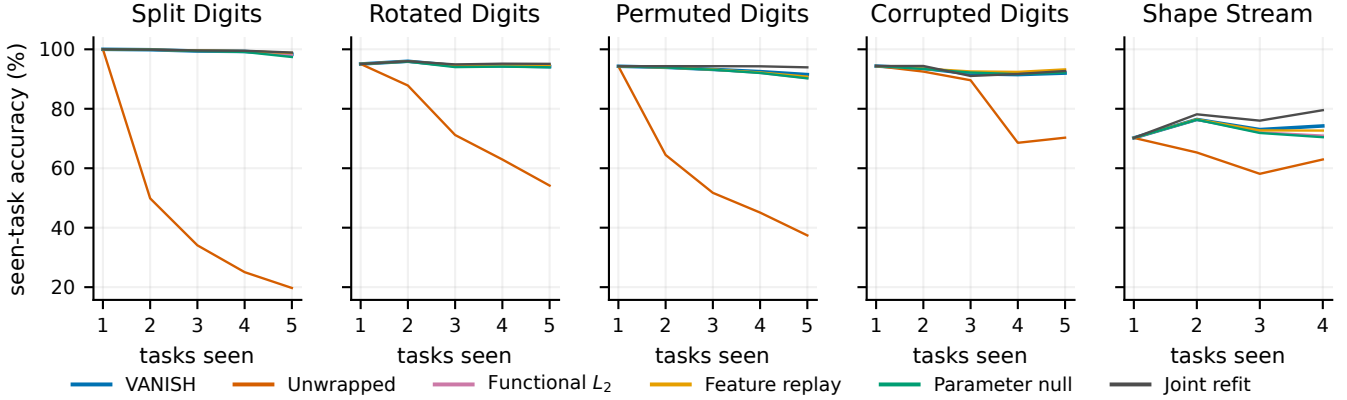


Figure 6: Average accuracy over all tasks seen so far, averaged over 20 seeds. Later safe increments may transfer to unseen old-domain examples while remaining exactly zero on the protected observations.

7. emit a manifest and SHA-256 checksums.

Every functional record contains the full lower-triangular accuracy matrix, elapsed seconds, parameter bytes, maximum cumulative anchor drift, patch diagnostics, hyperparameters, stream, method, and seed. The MLP records use the same schema. No table is hand-entered into the analysis files; this manuscript reports the generated CSV values. The reference code uses NumPy, SciPy, scikit-learn, and Matplotlib and requires no GPU or network download.

XII. CONCLUSION

VANISH changes continual learning from a rule about how parameters should move into a postcondition on what the realized function update is allowed to change. Its core operator subtracts the unique minimum-norm representer component observed by a protected contract. The resulting identity is exact for finite updates, independent of optimizer and patch magnitude, and extends from values to arbitrary bounded linear functionals.

The theory and experiments meet at the same observable. Across 600 matched runs, VANISH preserves high accuracy while driving protected-output motion about nine orders below replay. Under a nonlinear finite displacement, tangent safety fails by 55.25 while the function-space endpoint stays below 1.20×10^{-11} . Mixed value-and-derivative constraints and all task orders

close at machine precision. The hard-interpolation regime is not merely low-forgetting; it is exactly the offline solution. These results support a broader principle: optimize updates freely, then compile them through an explicit functional contract before deployment.

I. FULL PROOF OF OFFLINE EQUIVALENCE

Let $V_t = \text{span}\{k(x, \cdot) : x \in S_t\}$ and let P_{t-1} be the orthogonal projector onto V_{t-1} . For new sites X_t , define innovation representers

$$\tilde{k}_{t,x} = k(x, \cdot) - P_{t-1}k(x, \cdot), \quad x \in X_t. \quad (19)$$

Their Gram matrix is exactly the Schur complement $K_{t|<t}$ in (11). Strict positive definiteness of the full Gram matrix implies strict positive definiteness of every Schur complement. Hence the innovation representers are linearly independent.

Assume by induction that f_{t-1} is the minimum-norm interpolant of all earlier blocks and lies in V_{t-1} . Any function that retains the old values can be written $f_{t-1} + h$ with $h \in V_{t-1}^\perp$ after discarding components orthogonal to all data representers, which would only increase norm. The new constraints are

$$h(X_t) = Y_t - f_{t-1}(X_t) = R_t. \quad (20)$$

The representer theorem in $V_{t-1}^\perp$ gives $h = \sum_{x \in X_t} \beta_x \tilde{k}_{t,x}$ and the unique coefficients satisfy $K_{t|<t} \beta = R_t$. This is precisely (13).

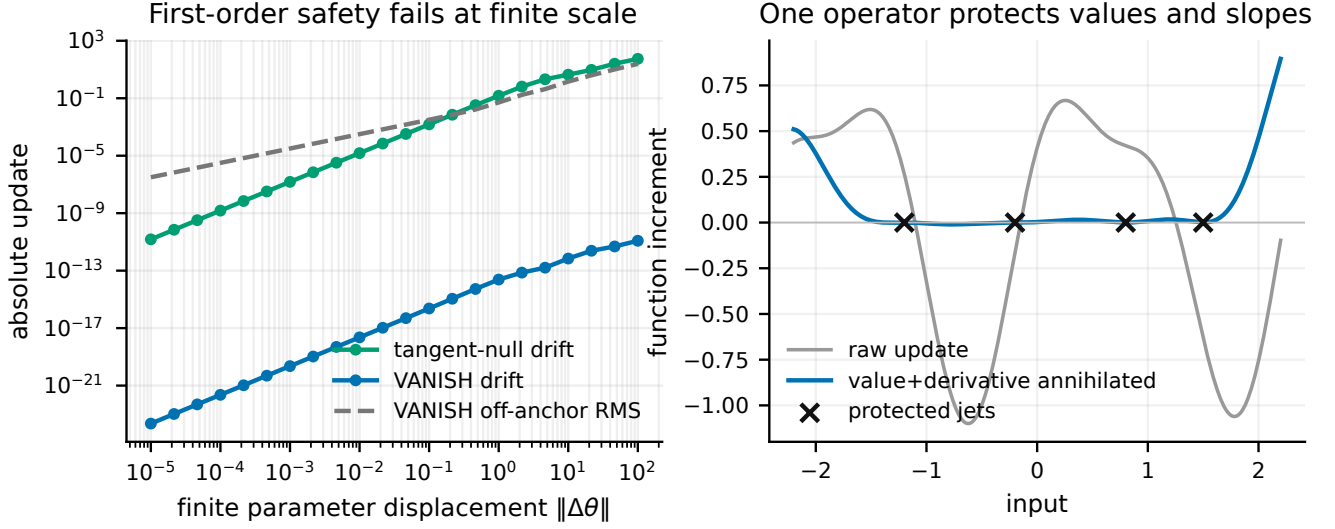


Figure 7: Mechanism-level falsification. Left: an exactly tangent-null direction is not safe after finite nonlinear displacement, whereas the realized update wrapped by VANISH remains at numerical zero on anchors. Right: the general operator simultaneously annihilates values and derivatives without erasing the update elsewhere.

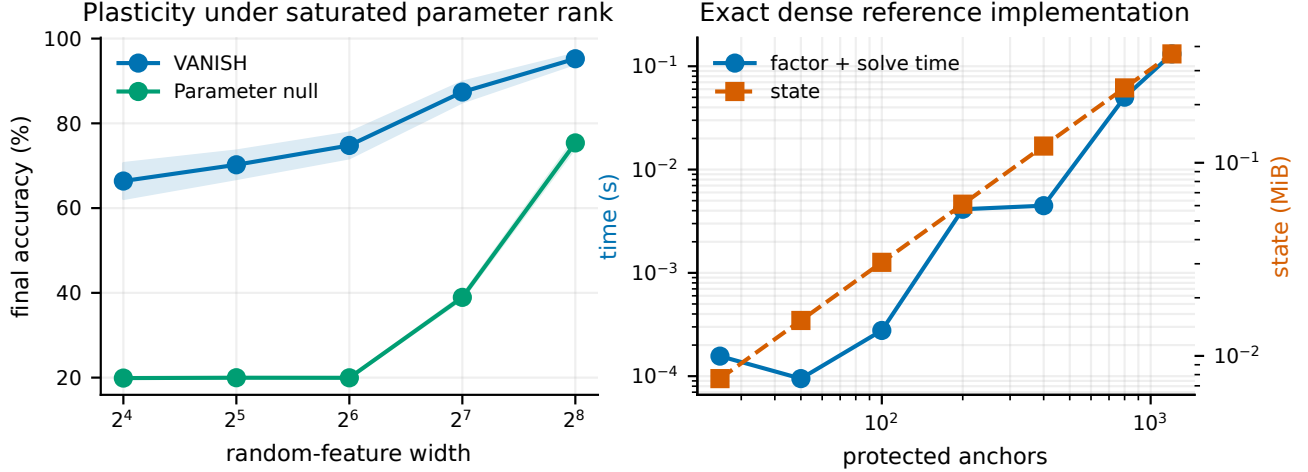


Figure 8: Left: function-space annihilation preserves plasticity when a finite parameter null space saturates. Shading is one standard deviation over 20 seeds. Right: measured dense reference time and stored state versus protected-anchor count.

Orthogonality gives $\|f_{t-1} + h\|_{\mathcal{H}}^2 = \|f_{t-1}\|_{\mathcal{H}}^2 + \|h\|_{\mathcal{H}}^2$, so the induction step is globally minimum norm subject to all constraints. At $t = T$, uniqueness identifies the result with the offline representer solution $K_{XX}^{-1}Y$.

For a different task order, the same proof ends at the unique minimum-norm interpolant of the same union of labeled sites. Therefore all orders yield the same function. Numerically, block Cholesky realizes exactly this orthogonalization; the permutation experiment tests the complete construction rather than only its values at training sites.

II. GENERAL VECTOR-VALUED CONTRACTS

Let outputs live in $\mathbb{R}^c$ and let $\mathcal{H}_K$ be a vector-valued RKHS with operator-valued kernel $K(x, z) \in \mathbb{R}^{c \times c}$. A functional may select a component, combine components, or differentiate them. Stack the corresponding representer as $\mathcal{C}^*$. Equations (6)–(8) remain unchanged, with G the block functional Gram matrix.

The experiments use the separable kernel $K(x, z) = k(x, z)I_c$, so one scalar Cholesky factor is reused for all classes. This reuse is both computationally efficient and exact.

If G is positive semidefinite, replace G^{-1} with the Moore–Penrose pseudoinverse after verifying consistency $\mathcal{C}u \in \text{range } G$. Then $I - \mathcal{C}^*G^\dagger\mathcal{C}$ is still the orthogonal projector onto $\ker \mathcal{C}$. The implementation instead removes exact duplicates through deterministic data generation and adds audited jitter for near dependencies.

III. METRIC DEFINITIONS AND STATISTICAL AUDIT

For T tasks, define

$$\text{FAA} = \frac{1}{T} \sum_{j=1}^T a_{T,j}, \quad (21)$$

$$\text{AIA} = \frac{2}{T(T+1)} \sum_{t=1}^T \sum_{j=1}^t a_{t,j}, \quad (22)$$

$$\text{BWT} = \frac{1}{T-1} \sum_{j=1}^{T-1} (a_{T,j} - a_{j,j}), \quad (23)$$

$$\text{Fgt} = \frac{1}{T-1} \sum_{j=1}^{T-1} \left(\max_{t \geq j} a_{t,j} - a_{T,j} \right). \quad (24)$$

The raw records also contain worst-task forgetting and mean diagonal accuracy. Confidence intervals use Student's $t_{0.975, n-1} s / \sqrt{n}$. Paired tests align methods by identical stream and seed, use one-sided alternatives matching the metric direction, and apply Holm correction across the complete generated family. We retain nonsignificant outcomes rather than filtering them from the CSV.

For numerical values spanning many orders, the figures use logarithmic axes. Table 1 reports $\log_{10}$ of the arithmetic mean drift per method and stream; the macro row uses the geometric mean of stream means. Raw, untransformed values remain authoritative.

IV. HYPERPARAMETERS AND COMPUTE

Table 4: Primary functional configuration.

Stream	Width	RBF γ	Ridge	Cap/task
Split Digits	768	.030	10^{-4}	220
Rotated Digits	1024	.060	10^{-4}	180
Permuted Digits	1024	.060	10^{-4}	180
Corrupted Digits	1024	.030	10^{-4}	180
Shape Stream	1024	.015	10^{-4}	240

Random feature weights use variance $2/d_{\text{in}}$ and biases are uniform on $[-\pi, \pi]$. One-hot targets are +1 for the class and -1 otherwise. Functional L_2 weight is 10; matched replay weight is 1. MLPs use ReLU, Adam, batch size 64, learning rate 2×10^{-3} , weight decay 10^{-4} , 35 epochs per task, and deterministic initialization. BLAS threads are capped at four for repeatability. The paper reports per-run timing stored by the same process rather than profiler extrapolation.

V. EXECUTABLE CONTRACT MAP

The four unit tests correspond one-to-one with the theory:

- `test_arbitrary_vector_update_is_zero_at_anchors` instantiates Theorem 1 for a five-output random neural patch and checks nonzero off-anchor RMS;
- `test_finite_update_beats_tangent_promise` verifies the nonlinear endpoint separation;
- `test_values_and_derivatives` constructs the mixed functional Gram matrix; and
- `test_hard_interpolation_is_order_equivalent` enumerates all 24 permutations for Theorem 6.

The tests use thresholds at least three orders looser than the recorded residuals, avoiding fragile equality to a single machine or BLAS build.

VI. STREAMING FACTORIZATION DERIVATION

The reference implementation refactorizes the protected Gram matrix to keep the code auditable. A production implementation can update the exact factor without changing a single prediction. Suppose the current matrix and its Cholesky factor are

$$G_m = L_m L_m^\top, \quad (25)$$

and a block of q functionals gives the enlarged matrix

$$G_{m+q} = \begin{bmatrix} G_m & B \\ B^\top & D \end{bmatrix}. \quad (26)$$

Solve $L_m Q = B$ and factor the Schur complement $D - Q^\top Q = L_q L_q^\top$. Then

$$L_{m+q} = \begin{bmatrix} L_m & 0 \\ Q^\top & L_q \end{bmatrix} \quad (27)$$

satisfies $L_{m+q} L_{m+q}^\top = G_{m+q}$. The update costs $O(m^2 q + m q^2 + q^3)$ and reuses all earlier work. Solves for c output channels perform two triangular substitutions with the same factor.

This block form also exposes the connection to Theorem 6. The Schur complement is the Gram matrix of representers after their projections onto the old span have been removed. A zero or ill-conditioned eigenvalue therefore has a direct interpretation: the new contract repeats, or nearly repeats, information already declared. Pivoting may remove the dependent row while preserving the same constraint set. Jitter instead changes the exact projector by a controlled numerical amount, which is why the implementation publishes the realized residual rather than silently treating it as zero.

For a solve $(G + \varepsilon I)\alpha = v$, the protected residual is

$$v - G\alpha = \varepsilon\alpha. \quad (28)$$

Hence $\delta_\infty \leq \varepsilon \|\alpha\|_\infty$ in exact arithmetic for the regularized system, plus backward error from factorization. This equality explains both the observed 10^{-13} – 10^{-9} range and why scaling v or worsening conditioning can raise it. The per-task audit measures the quantity users care about after both effects have occurred.

VII. EXPANDED AGGREGATE AUDIT

Table 5 reports metrics omitted from the compact main table. The matched raw generators have similar new-task accuracy near 90%, confirming that the unwrapped collapse is interference rather than a failure to learn each arriving task. VANISH retains that diagonal accuracy while replacing 51.79 points of macro forgetting with 1.69 points. Its mean runtime is lower than both matched replay variants in this CPU reference. Additional state stores anchors and correction coefficients, raising macro parameter-state size from 4.15 to 5.58 MiB.

The difference between mean accuracy forgetting and contract drift is visible here. Feature replay has slightly lower macro accuracy forgetting than VANISH by 0.09 points, yet it changes protected logits by nine orders more. Conversely, an unchanged logit contract does not forbid accuracy on an unprotected test example from moving. These are different observables and should not be collapsed into a single “forgetting” number.

Table 5: Macro audit across five streams. Accuracy quantities are percentages; time and state are per completed run.

Method	FAA	AIA	New	Forget	Sec.
Unwrapped	48.90	58.23	90.10	51.79	1.15
Functional L_2	89.28	90.12	90.17	1.84	1.60
Feature replay	90.05	90.56	90.62	1.60	1.58
Parameter null	88.84	89.90	89.89	1.98	1.43
VANISH	90.05	90.49	90.67	1.69	1.35
Joint refit	92.03	91.70	91.78	1.11	1.57

VIII. ANCHOR COVERAGE CERTIFICATE

Theorem 5 gives a computable bridge from finite anchors to unobserved inputs. For an RBF kernel, $p_S(x)$ is zero on S and decreases when x is well represented by nearby anchors. A deployment can therefore publish two certificates:

1. the exact algebraic residual $\delta_{\max}$ on the declared contract; and
2. a coverage statistic such as a high quantile of $p_S(x)$ on an unlabeled operational sample.

The first is independent of patch norm. The second combines with $\|u\|_{\mathcal{H}}$ through (10) to control off-contract motion. This separates an engineering choice—where to place anchors—from the mathematical mechanism that preserves them. Active selection can minimize the maximum power function, while class-balanced selection can target semantic coverage. Neither changes the annihilator.

For vector-valued separable kernels, apply the bound to each output or use the Euclidean vector RKHS norm to obtain

$$\|h(x)\|_2 \leq \|u\|_{\mathcal{H}_K} p_S(x). \quad (29)$$

If the classification margin at an old point is $M(x)$ and the bound is smaller than $M(x)/2$, the predicted class cannot change. Thus a functional certificate can be translated into a decision certificate wherever a margin and coverage bound are available.

IX. ARTIFACT FILE MAP

The source tree deliberately separates mechanism, experiments, and presentation. `src/vanish/core.py` contains the only preservation implementation. `experiments/run_suite.py` runs matched functional methods; `run_mlp_baselines.py` runs the conventional models; `stress_tests.py` contains the nonlinear, derivative, order, capacity, and scaling probes. `summarize.py` produces all CSV statistics, and `make_figures.py` reads raw results without rerunning a model. `tests/test_core.py` is the executable theorem boundary. The paper directory contains this source, its class file, bibliography, and vector figures.

The manifest records path, byte count, and SHA-256 for every packaged file. The outer PDF and ZIP use content-derived hash suffixes, so a changed byte necessarily changes the delivered name. This makes the version cited by a reviewer unambiguous even when later revisions share the semantic version.

REFERENCES

- [1] M. McCloskey and N. J. Cohen, “Catastrophic interference in connectionist networks: The sequential learning problem,” in *Psychology of Learning and Motivation*, vol. 24, pp. 109–165, Academic Press, 1989.
- [2] R. M. French, “Catastrophic forgetting in connectionist networks,” *Trends in Cognitive Sciences*, vol. 3, no. 4, pp. 128–135, 1999.
- [3] I. J. Goodfellow, M. Mirza, D. Xiao, A. Courville, and Y. Bengio, “An empirical investigation of catastrophic forgetting in gradient-based neural networks,” in *International Conference on Learning Representations*, 2014.
- [4] J. Kirkpatrick *et al.*, “Overcoming catastrophic forgetting in neural networks,” *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [5] F. Zenke, B. Poole, and S. Ganguli, “Continual learning through synaptic intelligence,” in *International Conference on Machine Learning*, pp. 3987–3995, 2017.
- [6] R. Aljundi, F. Babiloni, M. Elhoseiny, M. Rohrbach, and T. Tuytelaars, “Memory aware synapses: Learning what (not) to forget,” in *European Conference on Computer Vision*, pp. 139–154, 2018.
- [7] Z. Li and D. Hoiem, “Learning without forgetting,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 12, pp. 2935–2947, 2018.
- [8] X. Liu *et al.*, “Elastic weight consolidation done right for continual learning,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026.
- [9] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, “icarl: Incremental classifier and representation learning,” in *IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2001–2010, 2017.
- [10] D. Lopez-Paz and M. Ranzato, “Gradient episodic memory for continual learning,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [11] A. Chaudhry, M. Ranzato, M. Rohrbach, and M. Elhoseiny, “Efficient lifelong learning with a-gem,” in *International Conference on Learning Representations*, 2019.
- [12] M. Riemer, I. Cases, R. Ajemian, M. Liu, I. Rish, Y. Tu, and G. Tesauro, “Learning to learn without forgetting by maximizing transfer and minimizing interference,” in *International Conference on Learning Representations*, 2019.
- [13] A. A. Rusu *et al.*, “Progressive neural networks,” *arXiv preprint arXiv:1606.04671*, 2016.
- [14] A. Mallya and S. Lazebnik, “Packnet: Adding multiple tasks to a single network by iterative pruning,” in *IEEE Conference on Computer Vision and Pattern Recognition*, pp. 7765–7773, 2018.
- [15] J. Schwarz *et al.*, “Progress & compress: A scalable framework for continual learning,” in *International Conference on Machine Learning*, pp. 4528–4537, 2018.

- [16] H. Wang *et al.*, “Self-expansion of pre-trained models with mixture of adapters for continual learning,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.
- [17] M. Bennani, T. Doan, and M. Sugiyama, “Generalisation guarantees for continual learning with orthogonal gradient descent,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 8977–8989, 2020.
- [18] G. Saha, I. Garg, and K. Roy, “Gradient projection memory for continual learning,” in *International Conference on Learning Representations*, 2021.
- [19] S. Wang, X. Li, J. Sun, and Z. Xu, “Training networks in null space of feature covariance for continual learning,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 184–193, 2021.
- [20] M. K. Titsias, J. Schwarz, A. G. de G. Matthews, R. Pascanu, and Y. W. Teh, “Functional regularisation for continual learning with gaussian processes,” in *International Conference on Learning Representations*, 2020.
- [21] M. M. Derakhshani, X. Zhen, L. Shao, and C. G. M. Snoek, “Kernel continual learning,” in *International Conference on Machine Learning*, pp. 2621–2631, 2021.
- [22] T. G. J. Rudner, F. Bickford Smith, Q. Feng, Y. W. Teh, and Y. Gal, “Continual learning via sequential function-space variational inference,” in *International Conference on Machine Learning*, pp. 18871–18887, 2022.
- [23] N. Aronszajn, “Theory of reproducing kernels,” *Transactions of the American Mathematical Society*, vol. 68, no. 3, pp. 337–404, 1950.
- [24] B. Schölkopf and A. J. Smola, *Learning with Kernels*. MIT Press, 2002.
- [25] C. A. Micchelli and M. Pontil, “On learning vector-valued functions,” *Neural Computation*, vol. 17, no. 1, pp. 177–204, 2005.
- [26] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. MIT Press, 2006.
- [27] H. Kang *et al.*, “Do your best and get enough rest for continual learning,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.
- [28] Y. Liu *et al.*, “Lora subtraction for drift-resistant space in exemplar-free continual learning,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.
- [29] W. Li *et al.*, “A faster path to continual learning,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026.
- [30] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [31] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.